pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Case Study: Ridge, Lasso, and the Spread Cross-Section

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 1 closes: ridge, lasso, and a case study

Day 1	One feature, the split
Day 2	Dummy variables
Day 3	Coefficients, p-values
Day 4	The log of OAS

Today's result, up front

On this benchmark, ridge and lasso do not beat plain regression: the three test MAEs lie within 0.09 bp of each other.

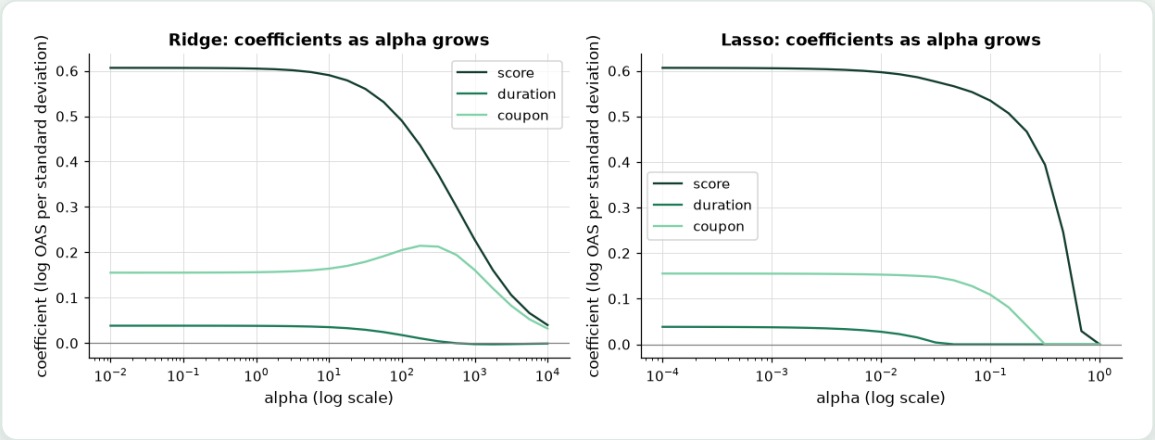
Today also brings the Pipeline: every fitted step on the training rows only.

821 invented bonds of 150 issuers, spreads from the course's own recipe. Sector plays no part in it.

Shrinking coefficients: ridge and lasso

Model	Penalty added to the errors
OLS	none
Ridge	alpha x sum of squared coefficients
Lasso	alpha x sum of absolute coefficients

Coefficient	OLS	Ridge 10	Lasso 0.05
score	0.6065	0.5904	0.5640
duration	0.0384	0.0352	0.0000
coupon	0.1553	0.1641	0.1383



alpha sets the penalty's strength: chosen before the fit, a hyperparameter.

Why scaling now matters

Coupon in percent or in bp: OLS gives the same fitted spreads. A penalty charges a coefficient by its size, and its size depends on the unit.

0.0 bp

OLS, not scaled: largest gap between the two fits

21.7 bp

Ridge alpha 100, not scaled: largest gap between the two fits

0.0 bp

Ridge alpha 100, scaled: largest gap between the two fits

StandardScaler: (x - mean) / population standard deviation, Excel's STDEV.P. It learns two numbers per column. From which rows?

Scaled, every feature is in standard deviations, whatever unit it started in.

Pipeline and ColumnTransformer

OneHotEncoder

rating, sector

learns: levels, less A, Consumer

→

StandardScaler

4 numbers

learns: mean, standard deviation

→

Model

30 columns

learns: coefficients, intercept

```
pipeline = build_pipeline(LinearRegression()).fit(X_train, y_train)   # one fit, every step
```

Which rows did the scaler learn from?

Training rows' means: True. All 821 rows' means: False.

mlkit.named_coefficients

Every coefficient with its name. 20 tests pass.

Choosing alpha inside the training rows



Four folds fit, one scores, five times. The test rows stay put away.

Ridge, 13 alphas `0.0316228`

RidgeCV: the same, True

Lasso, its own grid `0.000581227`

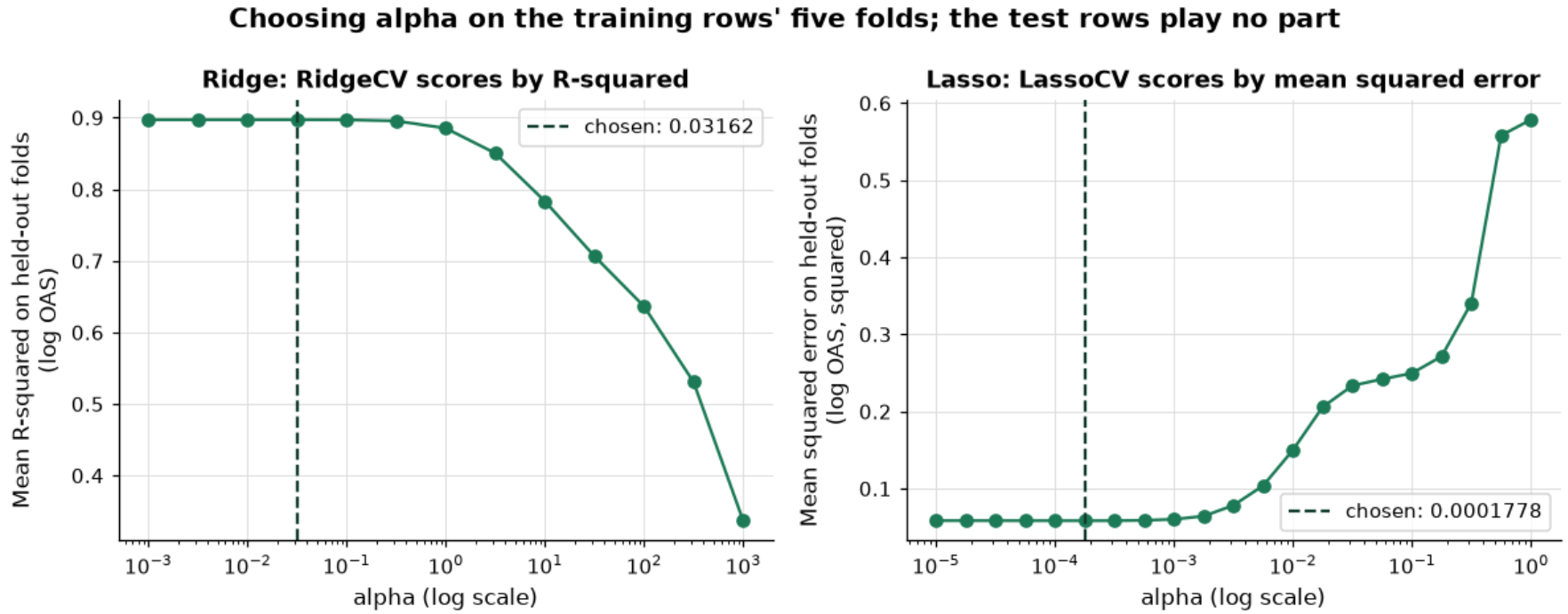
the smallest on its grid: True

Lasso, wider grid `0.000177828`

number 6 of 21: inside

A choice at the edge of a grid: widen the grid. Both alphas are small: little penalty wanted here.

Choosing alpha, in one picture

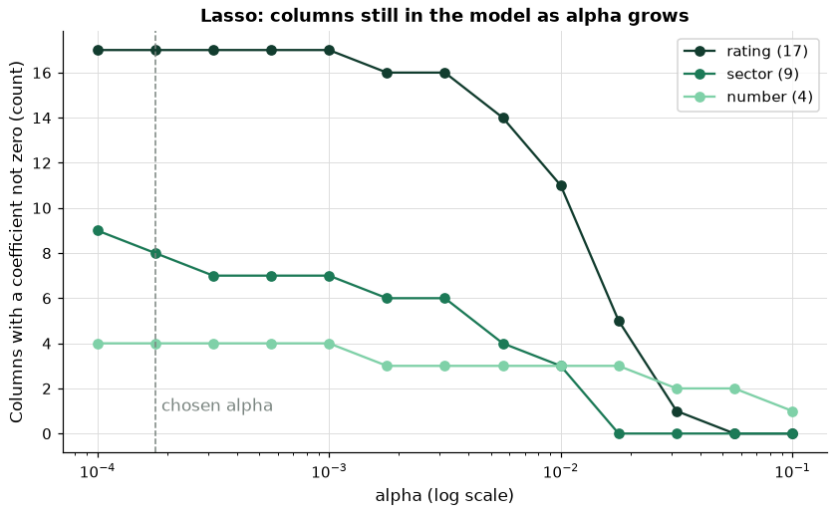


Both curves are flat at the small end; the chosen alphas, 0.0316228 and 0.000177828, sit there. The test rows play no part.

Lasso's zeros, and why on these bonds

At the chosen alpha: 1 of 30 at zero, sector_Technology

At alpha 0.003: 3 of 9 sectors, 0 of 17 ratings, and years at zero



Sector

plays no part in the recipe: noise, gone first

Years

duplicates duration (correlation 0.95): one is kept

Rating

carries the recipe's biggest term: gone last

A zero describes these rows at this alpha. It is not a finding that a feature is unrelated to spreads.

The week 1 case study

How much of the spread cross-section do rating, sector, duration, years, coupon, and size account for, on 206 test bonds the model never saw? Does a penalty help?

1. Three models, same rows: OLS, ridge, lasso

2. Test rows once: MAE in bp, R-squared beside

3. How much one split moves a number

4. The random split against the split by issuer

5. Fitted rating effects beside the generator

6. What the model shows, and where it fails

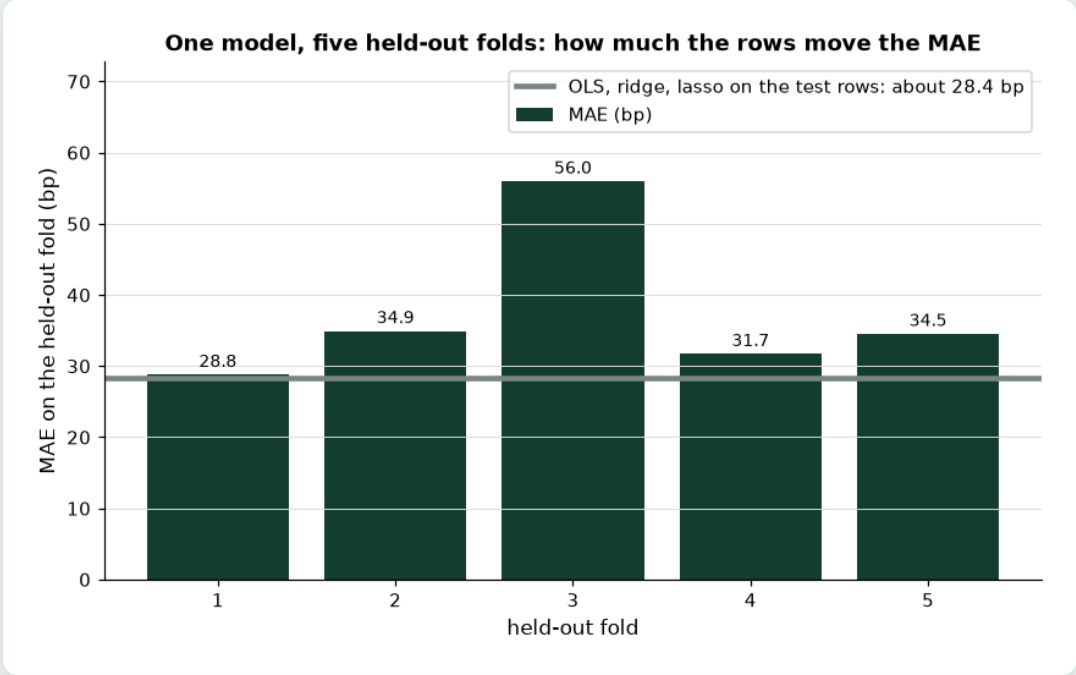
Synthetic bonds: every answer describes the course's recipe, and can be checked against it.

The comparison, read honestly

Model	R-squared (log OAS)	MAE (bp)	RMSE (bp)
OLS	0.8969	28.4	65.9
Ridge	0.8965	28.4	66.2
Lasso	0.8954	28.4	66.3

0.09 bp between the three test MAEs; five folds of one model: 28.8 to 56.0 bp

Why: 615 rows fit 30 coefficients, and the recipe is additive in logs, the model's own shape.



One model, five folds: a range of 27.2 bp. The grey line is all three models at once.

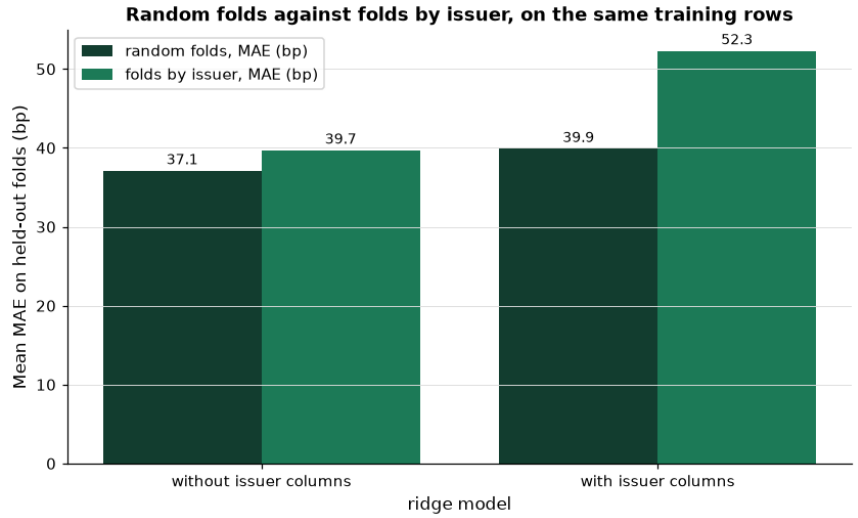
Where penalties earn their place: many features, few rows; features that move together (day 11).

The random split against the split by issuer

GroupShuffleSplit, groups = ticker

190 test bonds of 38 issuers; no issuer on both sides. Test MAE: OLS 46.2, ridge 46.1, lasso 46.2 bp.

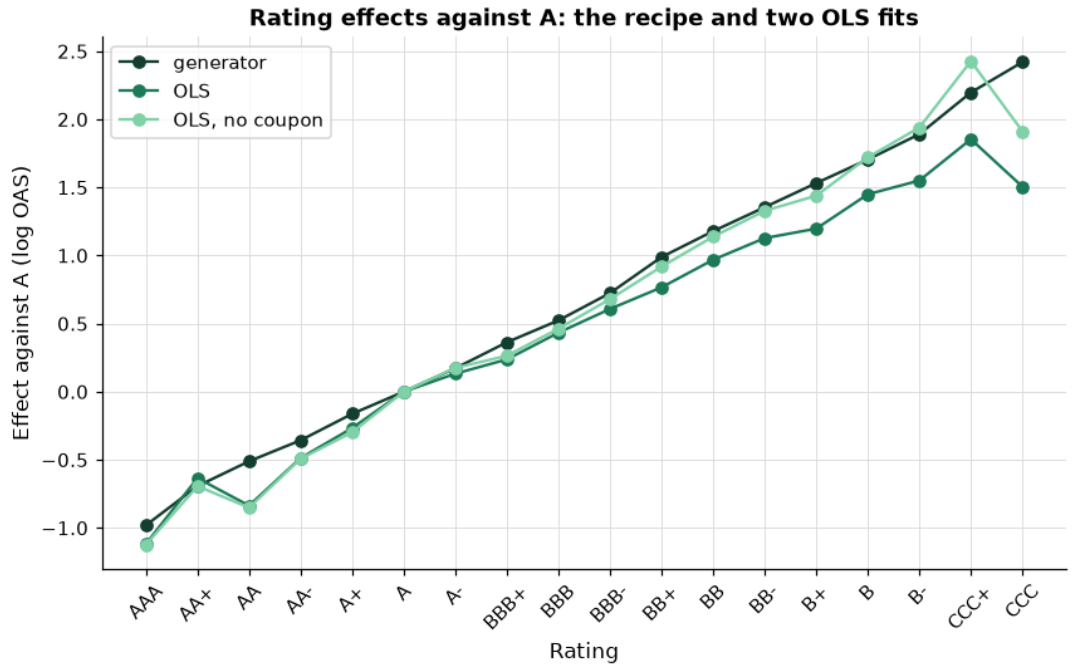
6 warnings: a fold held a rating its training rows lacked.



When a feature identifies the issuer, split by issuer.

The two test sets hold different bonds, so compare ways of validating on the same rows.

The fitted rating effects beside the generator



8 of 17

within 0.2 with coupon

14 of 17

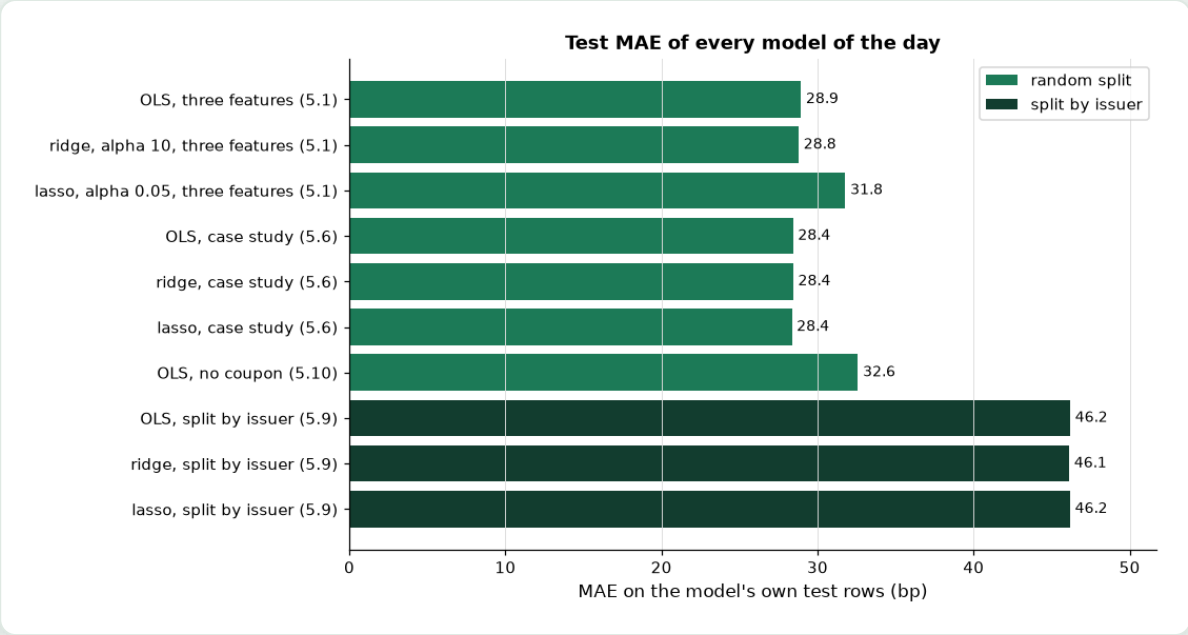
within 0.2 without

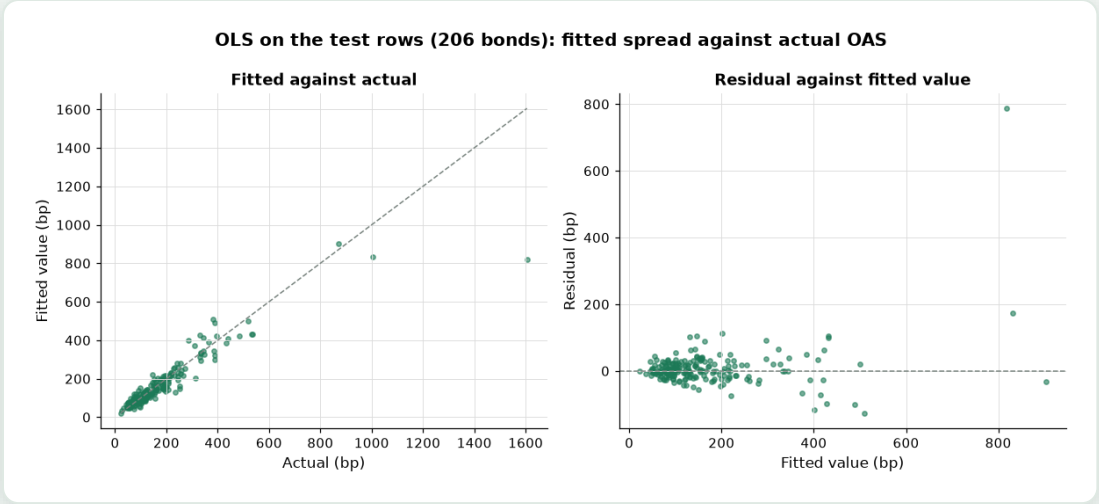
Why the coupon pulls them down

The recipe sets each coupon near the bond's own yield: Treasury plus its OAS, plus noise. Correlation with log OAS: 0.77.

Without the coupon, test MAE 32.6 bp, against 28.4: better at fitting, worse at recovering the recipe.

Every model of the day, and where the misses are





Random split: 28.4 to 32.6 bp for every model. By issuer: about 46 bp. The split moves the number more than the model.

What the model shows, and where it fails

What it shows

90% of the variation in log OAS on test rows; MAE 28.4 bp. Mostly the rating, some the coupon. Ridge and lasso: the same.

Wide spreads

MAE 47.0 bp HY, 18.9 bp IG; the five largest misses: 22% of all error

New issuers

split by issuer, MAE 46.2 bp

One split

five folds: 28.8 to 56.0 bp

Thin ratings

AA+, AA, CCC: one issuer each

Synthetic bonds. A fitted spread is the model's spread for these characteristics: not a fair value, and no basis for a decision.

Excel side by side: ridge in one formula

Excel side by side (tested in Excel)

`=STDEV.P(C2:C616)`	3.2209, duration
`=STANDARDIZE(B2,G$1,G$2)  (K2, fill)`	a scaled feature
`=AVERAGE(A2:A616)`	intercept 4.8776
`=TRANSPOSE(MMULT(MINVERSE(Z'Z+10*MUNIT(3)),Z'(y-mean)))`	0.5904, 0.0352, 0.1641
`the same on STDEV.S columns`	0.5908, 0.0352, 0.1643: differ
`=STANDARDIZE(B2,train!G$1,train!G$2)`	test rows, training numbers
`=AVERAGE(ABS(A2:A207-I2:I207))`	MAE 28.8 bp

Every number within 1e-9 of Python. Excel has no lasso: it has no formula, only a search.

Save the week: a tag on the last commit

```
abc1234 (HEAD -> main, tag: week1) Experiment log: ridge and lasso, alpha chosen by cross-va...
abc1234 Experiment log: OLS on log OAS with rating, sector, duration, years, coupon, amount
abc1234 named_coefficients, with three tests: the coefficients of a pipeline, labelled
```

`git diff week1~1 week1 -- results.csv`

```
 model,features,train_rows,test_rows,test_r2,test_mae_bp,test_rmse_bp
 ols_log_oas,"rating, ...",615,206,0.8969,28.4333,65.8637
+ridge_log_oas,"rating, ...",615,206,0.8965,28.4438,66.1527
+lasso_log_oas,"rating, ...",615,206,0.8954,28.3562,66.3068
```

A tag never moves. The diff is the case study's result in four decimals.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: StandardScaler is fitted on all 821 rows before the split (821 rows learned, 28.8 bp).

Code that runs is not the same as code that is right.

What you can do now

- Ridge and lasso add a penalty; alpha sets its strength
- A penalty makes scaling matter: fit it inside a Pipeline
- Choose alpha inside the training rows; widen a grid at its edge
- Lasso drops sectors and years first, as the recipe says
- Here, ridge and lasso do not beat plain regression

Week 2, Day 1 Logistic regression and odds on the UCI card file